Phase 7 developer handoff

Phase 7C discovery-led proposal and synthetic validation (10 October 2026)

The customer does not need to provide a target schema or complete ownership rules before discovery. `data_discovery.py` parses the authorized pinned task snapshot for supported SQL DDL, Liquibase XML, Java JPA annotations, Spring Data repository methods and table-level SQL patterns. It emits entities, columns, keys, foreign keys, read/write paths, parser coverage and evidence references `{path, sha256, line, parser}`. Ownership candidates are package/domain matches and are labeled as hypotheses with confidence; conflicting writers and shared access remain unresolved. Runtime database graph observations are context-only because table identity may not be established.

`propose_target()` creates a logical or `SQLITE_TEST` target even when source schema is absent; absent evidence becomes explicit questions and an incomplete proposal, not a discovery blocker. Proposal includes target tables and candidate service boundaries, mappings and conversion candidates, alternatives, ordered migration steps, unresolved risks, optional customer-authored invariants, and bindings for repository commit, complete source manifest, architecture-version fingerprint/artifact, source-schema fingerprint, target-schema hash and mapping version. `revise_data_migration()` writes an immutable plan revision and advances the mapping version.

The APIs are `POST /api/v1/transformation/tasks/{id}/data-discovery`, `POST .../data-migration-proposals`, `POST .../data-migration-proposals/{plan_id}/revise`, `POST .../{plan_id}/review`, and `POST .../{plan_id}/synthetic-test`. Every operation checks current task source grants, policy, pinned baseline and architecture version. Invariants must reference discovered tables and (for `UNIQUE`/`NOT_NULL`) columns. The separate human reviewer must hold `SECURITY_ADMIN`, differ from task author, decide every ownership claim and acknowledge all residual soft risks. Approval binds both the original proposal fingerprint and the fingerprint of the reviewed document including those decisions; it authorizes only synthetic test execution.

`migration_test_runner.py` accepts only an in-memory fixture shape—never a connection string. It creates source and proposed target tables in SQLite `:memory:`, copies converted rows and verifies row counts, PK/unique constraints, foreign-key integrity, supported types, and `ROW_COUNT_EQUAL`, `UNIQUE`, `NOT_NULL`, `FOREIGN_KEY_VALID` invariants. Raw fixture rows are not

stored. Run records keep result/check fingerprints, state and limitations only. Failure rolls the in-memory transaction back and process closure discards it; backup/restore is explicitly reported untested.

`0008_data_modernization.py` adds tenant/task-bound discovery, proposal and run records. `DataMigrationWorkspace.tsx` renders evidence, parser coverage, gaps, generated schema, mappings, owner decisions, risk acknowledgments, fixture form and test output. The older `/data-migration-plan` interface remains for compatibility but is no longer the primary tab workflow; it remains a preview only.

The current proposal generator uses deterministic evidence rules; it does not invoke an LLM or claim semantic inference of business rules. If model-assisted suggestions are added later, preserve citations to repository evidence, keep generated statements as hypotheses, and require the same independent review gate. Current scope is local synthetic validation, not production data migration. The parser is deliberately bounded and does not prove complete schema/query discovery for every dialect, ORM, stored procedure, dynamic SQL construction or external runtime data path. SQLite checks do not establish Concord/PostgreSQL compatibility. No production adapter, credential handling, live or representative customer data path, cutover, backup restore drill, automatic recovery guarantee, or production approval action exists. Keep production execution disabled until a database-specific disposable integration and separate destructive-action controls are designed and reviewed.

9 October 2026 implementation continuation

The task Validation tab now supports a measured expected-improvement review, candidate version publication, and final human approval. The service measures only plan-relevant static evidence (for example, cross-domain dependency edges or class cycles), preserves the baseline class inventory, and records a deterministic fingerprint. A candidate `ArchitectureVersion` is created only after all required gates pass and is bound atomically to the task. Final approval requires the latest fresh validation, an authorized security administrator distinct from the task author and prior reviewers, and stores the validation/gate fingerprints. The routes are `POST /api/v1/transformation/tasks/{id}/expected-impact/review`, `/candidate-version`, and `/final-approval`.

Packaged dependency scanning locates embedded archives in Spring Boot JARs, WARs, and generic `lib/` JARs, validates archive paths, caps enumeration at 1,000 libraries and 400 MiB expanded, extracts libraries to temporary storage, then scans them with Trivy offline. The vulnerability database cache defaults to `.runtime/trivy-cache` on the project drive and is mounted read-only at runtime. External classpaths, mounted libraries, and container-provided runtime libraries are not covered; such cases remain partial and cannot pass a complete-coverage approval gate.

The scanner's provisioning and end-to-end run are still blocked by the Docker disk image exhausting C: during Java database download. The partial vulnerability database is on D:, but the Java database and dependency image ID were not produced. Do not mark dependency evidence PASS until the image and both fresh databases are provisioned and a real scan succeeds. A final small adjustment moved Trivy's global `--cache-dir` before the subcommand to ensure downloads/scans use the mounted cache; revalidate it after Docker storage is repaired.

Latest test evidence at that earlier checkpoint: the bundled-Python focused suite passed **25 tests** in 95.22 seconds, with two dependency deprecation warnings. Phase 7B and 7C progress is updated in the current sections above and below.

10 October 2026: Phase 7B extraction-boundary preview

`backend/app/transformation/extraction.py` is a pure, deterministic boundary planner. `POST /api/v1/transformation/tasks/{id}/extraction-plan` binds its preview to the current approved architecture version, task source manifest, selected domain, and contract-review digest. It uses version-reviewed domain membership rather than silently reverting to scanner-discovered assignments. The task revision/event log records each preview; `service.view` marks it stale if the source manifest, contracts, ownership review, task state, or latest application version changes.

The preview reports target classes/packages, conservative source-path candidates, incoming/outgoing static edges, cycles crossing the target boundary, incomplete snapshots, unresolved source names, and required behavioral/data ownership reviews. Filenames are only package/type suffix candidates; the preview does not parse source into an AST. The preview is always `PREVIEW_ONLY` with `execution_eligible=false`; it does not generate changes, move files, execute code, or authorize extraction. It uses existing task/version storage and needs no migration.

This is the first 7B implementation slice, not module/service extraction. Generation, target build configuration, integration contracts, runtime/data ownership gates and isolated candidate execution remain future slices. Docker-backed builds remain deferred by the user.

Delivered checkpoint

`backend/app/transformation/validation.py` implements a pure deterministic evidence comparison, rule version `transformation-validation-1`. `service.validate` authorizes source access, verifies current planning context, expected revision and workspace integrity, then persists the result under the existing task JSON plus a history event. No schema migration is required.

`POST /api/v1/transformation/tasks/{id}/validate` accepts only `expected_revision` and a review `note`. Clients cannot submit passing gate results. It requires a staged task with no active

build. Existing tenant, source grant, policy and analysis-write checks apply. Comparison does not execute code or invoke models.

Candidate build records now include the staged changeset fingerprint. Old records without this binding fail comparison; rebuild the candidate. The existing source input digest and artifact hash remain authoritative build metadata. Latest failed attempts cannot be masked by older successful runs.

The result records selected build IDs, exact changeset fingerprint, candidate source digest, contract results, explicit gate states and a canonical comparison fingerprint. Unchanged results are idempotent after revision checking. Task reads recompute input comparison and expose `CURRENT`/`STALE`; paused, building and rolled-back tasks cannot present the old result as current. Historical results remain in task history.

Gates and limits

- Complete source snapshot and baseline/candidate build hashes must match captured manifests.
- Baseline artifact must match the analyzed artifact. The latest candidate must bind to the staged patch.
- Images must match. This is container-image comparability, not a claim that workloads, clocks or runtime environments are fully deterministic.
- Every registered contract must retain its reviewed document hash and unchanged captured test file. Both reports must contain exactly one passing case with the exact name. Duplicate/parameterized ambiguous names fail conservatively.
- Candidate inventory must preserve baseline test case multiplicities and have no failed/skipped cases. Additional passing tests are allowed.
- Architecture re-analysis requires a completed candidate analysis and eligible graph diff bound to the current build, artifact and changeset. Its gate confirms analysis execution/comparability, not expected-impact acceptance. Generated-code security stays `NOT_RUN`. Overall approval eligibility is always false in this checkpoint.

Build reports originate in customer-controlled builds. This comparison validates recorded evidence consistency, not report authenticity against malicious build logic. It cannot prove semantic equivalence, detect deliberately forged reports or establish full scenario coverage. Existing sandbox isolation remains necessary. A secret filter is not SAST or dependency vulnerability scanning.

At this checkpoint, one bounded AST recipe and the Java security adapter were implemented; complete runtime dependency coverage and final review remained pending. Phase 7B extraction generation and Phase 7C's synthetic migration test were not yet implemented. Current Phase 7C scope and limits are documented at the top of this guide; never change missing scanner evidence to PASS to unblock a demonstration.

Isolated security adapters

`POST /api/v1/transformation/tasks/{id}/security-scan` requires current source authorization, planning context, revision, workspace integrity and execution policy. The latest passing candidate build must bind to the staged patch and current source. Captured artifact path, size and SHA-256 are checked before execution. A durable `BuildRun(kind=SECURITY)` records intent before releasing the process lock. Reauthorization after execution prevents accepting results after policy changes or pause. Interrupted attempts recover as failed; retries invalidate previous results immediately.

`security.py` invokes SpotBugs 4.10.4 with FindSecBugs 1.14.0 using the fixed security-only filter under `infra/docker/security`. The image has no customer build configuration. XML is bounded, rejects DTD/entities, requires analyzed classes and consistent finding counts, and treats missing classes/errors as incomplete. Only finding identifiers and class names are retained, together with report digest, tool versions, image and artifact hashes. Runtime uses the existing read-only, non-root, offline resource-limited sandbox. The public tool configuration follows the FindSecBugs CLI documentation and SpotBugs CLI.

`dependencies.py` uses Trivy 0.75.0, with separately provisioned public vulnerability/Java databases. It disables customer configuration/ignore files, remote dependency lookup, telemetry and database updates at runtime. It rejects database timestamps older than seven days, malformed reports and empty package inventories. Its scope is explicitly `PACKAGED_JAVA_ONLY`; it cannot certify externally supplied runtime dependencies. See Trivy offline requirements. Findings include unfixed vulnerabilities. `GENERATED_CODE_SECURITY` remains blocked until complete runtime dependency coverage is implemented.

Build the two dedicated image contexts with immutable base digests, then configure the locally inspected image IDs in `SECURITY_SANDBOX_IMAGE` and `DEPENDENCY_SANDBOX_IMAGE`. Provisioning downloads public tools/databases without customer inputs. Application scans never pull images or download updates. Refresh the dependency image regularly; retain pinned IDs with verification evidence. No schema migration is needed: attempts and evidence use existing tenant-scoped records and task JSON.

Bounded AST recipe (6 October 2026)

`INTRODUCE_INTERFACE` uses JavaParser 3.28.2 in the immutable build image. The tool accepts one captured implementation path, one declared new interface path, an interface identifier and up to 20 unique method names. It requires one top-level class and exactly one public non-static method for every name, rejecting overload ambiguity. It produces only two UTF-8 outputs: the modified implementation and new interface. Each is capped at 256 KiB and flows through the existing secret filter, exact diff, approval, staging and rollback controls.

`POST /api/v1/transformation/tasks/{id}/recipes` requires the current revision and source authorization. Both paths must be in `allowed_files`; generation only runs against the unchanged pinned baseline. The resulting ChangeSet records recipe/parser versions, immutable image ID, input SHA-256, method selection and output paths. It remains `PROPOSED`; the container cannot write the task workspace.

The server-owned helper source is under `infra/docker/refactor`. `build-sandbox.Dockerfile` compiles it in a Maven stage and copies only the helper JAR and runtime dependency into the final image. Runtime execution remains non-root, read-only, capability-free, PID/memory/CPU bounded and network-disabled. Recipe version 1.0.1 enforces matching public type/file names and sibling output paths, copies imports and rejects unsupported generic/nested scopes, static imports and overloaded selections. The current build image is `sha256:9847fa2c085e6b1dfc3a7921d16184ee1925c20fc50b50d76fd8433ba8acfd81`.

This recipe introduces an abstraction seam but does not rewrite consumers or prove reduced coupling. It rejects ambiguous cases instead of attempting symbol inference. Full transformation acceptance still depends on the reviewed patch, candidate compile/tests, unchanged contract tests, static re-analysis, expected-impact review and security results.

Candidate capture and re-analysis (6 October 2026)

Candidate builds capture a bounded JAR/WAR in the task's `artifacts/<build-id>` folder. Capture uses exclusive creation, rejects linked host storage, validates the archive with the existing ingestion validator, and preserves pre-existing files on a collision. Invalid newly created archives are removed. Absolute capture paths stay in internal build metadata and are omitted from task API build views. Customer build logic and reports remain untrusted; capture consistency is not proof that a malicious build produced truthful reports.

`POST /api/v1/transformation/tasks/{id}/reanalyze` uses `expected_revision` and `note`. It rechecks current source authorization, planning context, snapshot integrity, successful latest candidate build and patch/source/artifact bindings. It shares the ordinary scan admission lock and queue limit, creates tenant-owned Artifact and AnalysisRun records, and submits the existing AnalysisWorker. Artifact manifests identify the task/build so ordinary application analysis cannot accidentally select them. Workers recover queued/running scans through the existing recovery mechanism. No customer code is executed by static scanning.

Call the endpoint again after the scan completes to finalize comparison. Completed refreshes are idempotent. A changed build requires a new task in this initial implementation. Failed scans record a stable failure and do not retry automatically. Candidate captures remain in task storage; operator retention management is still required.

`architecture.compare` compares fully qualified class identities and directed typed dependency triples, SCC membership, finding identity/severity, domain assignments and summary snapshots. It

reports added and removed entities without guessing renames. Scanner name/version, engine version and configuration must match. Missing engine provenance is inconclusive. Both result digests and the diff fingerprint are persisted in task lineage. The validation fingerprint incorporates the architecture record, making earlier comparisons stale when re-analysis evidence changes.

This creates a task-linked immutable analysis snapshot, not a new canonical ArchitectureVersion or reviewed evolution projection. Candidate version publication, reviewed entity correspondence, expected-impact approval and runtime scenario comparison are still required for the full Phase 7 acceptance chain.

Domain comparisons currently use scanner-discovered assignments from each static analysis. A version's manual domain reviews are not reapplied to candidate classes. The edge delta covers internal application dependencies; external-library references and detailed metric-by-metric deltas are not yet included.

Verification

`backend/tests/test_transformation_validation.py` exercises negative evidence cases and the authenticated task API, including stale revisions, idempotency, rebuild invalidation and rollback. It uses mocked build results and does not claim a real customer refactor. Existing transformation tests cover source isolation and access controls. See `PHASE-7-VERIFICATION.md` for executed checks.